End-to-End Application Development & Local Deployment

Session Objective

Build and run a simple **Message Processor** application using:

- Git — Source control
- GitHub — Remote code repository
- Streamlit — User interface
- FastAPI — Backend/API
- Docker — Application packaging and local deployment

Final workflow

1. User
2. ↓
3. Streamlit UI
4. ↓
5. FastAPI API
6. ↓
7. Process Message
8. ↓
9. Response
10. ↓
11. Streamlit UI

Overall technology workflow:

12. Git → GitHub → Streamlit → FastAPI → Docker → Local Application

---

1. Prerequisites

Install/verify:

- Python 3.10+

- Git
- VS Code or another code editor
- Docker Desktop
- GitHub account

> Windows learners: use **PowerShell** for the session unless specifically using the Windows CMD commands provided below.

---

2. Project Structure

The project will eventually look like:

```
message-processor/
│
├── app.py
├── api.py
├── requirements.txt
├── Dockerfile
├── start.sh
└── README.md
```

---

3. Verify Python

Mac

```
python3 --version
```

PowerShell

```
python --version
```

If `python` does not work:

```
py --version
```

Windows CMD

24. python --version

Or:

25. py --version

---

4. Verify Git

All platforms:

26. git --version

---

5. Create the Project

Mac

27. mkdir message-processor
28. cd message-processor

PowerShell

29. mkdir message-processor
30. cd message-processor

Windows CMD

31. mkdir message-processor
32. cd message-processor

---

6. Check Current Directory

Mac

33. pwd

PowerShell

34. pwd

Windows CMD

35. cd

---

7. Create Initial Files

Required files:

36. app.py
37. requirements.txt
38. README.md

Mac

39. touch app.py requirements.txt README.md

PowerShell

40. New-Item app.py -ItemType File
41. New-Item requirements.txt -ItemType File
42. New-Item README.md -ItemType File

Windows CMD

43. type nul > app.py
44. type nul > requirements.txt
45. type nul > README.md

---

8. Verify Files

Mac

46. ls

PowerShell

47. ls

or:

48. Get-ChildItem

Windows CMD

49. dir

Expected:

50. app.py
51. README.md
52. requirements.txt

---

9. Initial Git Setup

Initialize Git

All platforms:

53. git init

Check status

54. git status

---

10. Initial Application

app.py — Initial Version

55. print("Message Processor Application")

README.md

56. # Message Processor
57.
58. A simple application demonstrating:
59.
60. Git → GitHub → Streamlit → FastAPI → Docker

requirements.txt

61. streamlit

---

11. Stage and Commit

Stage files

All platforms:

62. git add .

Check status

63. git status

Commit

64. git commit -m "Initial project setup"

View commits

65. git log --oneline

---

12. GitHub Repository

Create an empty GitHub repository:

66. message-processor

Do not initialize it with:

- README
- `.gitignore`
- License

because these are already being created locally.

---

13. Connect Local Repository to GitHub

Replace the URL below with your own GitHub repository URL.

All platforms:

67. git remote add origin https://github.com/USERNAME/message-processor.git

Check remote:

68. git remote -v

Set branch:

69. git branch -M main

Push:

70. git push -u origin main

---

14. Demonstrate a Git Change

Update `app.py`:

71. print("Message Processor Application")
72. print("Version 2")

Check changes:

73. git status

See exact changes:

74. git diff

Stage:

75. git add app.py

Commit:

76. git commit -m "Update application message"

Push:

77. git push

---

15. Create Python Virtual Environment

A virtual environment keeps project dependencies isolated.

Mac

78. python3 -m venv .venv

PowerShell

79. python -m venv .venv

Windows CMD

80. python -m venv .venv

---

16. Activate Virtual Environment

Mac

81. source .venv/bin/activate

PowerShell

82. .venv\Scripts\Activate.ps1

Windows CMD

83. .venv\Scripts\activate.bat

After activation, you should see:

84. (.venv)

at the beginning of the terminal prompt.

---

17. Windows PowerShell Activation Issue

If PowerShell blocks activation, run:

85. Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser

Then:

86. .venv\Scripts\Activate.ps1

---

18. Install Streamlit

Mac

87. pip install streamlit

If required:

88. python3 -m pip install streamlit

PowerShell

89. pip install streamlit

Or:

```
python -m pip install streamlit
```

Windows CMD

```
pip install streamlit
```

Or:

```
python -m pip install streamlit
```

---

19. Streamlit Application

Replace `app.py` with:

```
import streamlit as st

st.title("Message Processor")

message = st.text_input("Enter your message")

if st.button("Process Message"):
    st.write("You entered:")
    st.write(message)
```

---

20. Run Streamlit

Mac

```
streamlit run app.py
```

Alternative:

```
python3 -m streamlit run app.py
```

PowerShell

104. streamlit run app.py

Alternative:

105. python -m streamlit run app.py

Windows CMD

106. streamlit run app.py

Alternative:

107. python -m streamlit run app.py

Open:

108. http://localhost:8501

Test with:

109. hello world

---

21. Commit Streamlit Application

Stop Streamlit with:

110. Ctrl + C

Then:

111. git add .

112. git commit -m "Add Streamlit user interface"

113. git push

---

22. FastAPI Backend

Update `requirements.txt`:

```
streamlit
fastapi
uvicorn
requests
```

Install:

Mac

```
pip install fastapi uvicorn requests
```

Or:

```
python3 -m pip install fastapi uvicorn requests
```

PowerShell

```
pip install fastapi uvicorn requests
```

Or:

```
python -m pip install fastapi uvicorn requests
```

Windows CMD

```
pip install fastapi uvicorn requests
```

Or:

```
python -m pip install fastapi uvicorn requests
```

---

23. Create api.py

Mac

124. touch api.py

PowerShell

125. New-Item api.py -ItemType File

Windows CMD

126. type nul > api.py

Add:

```
127. from fastapi import FastAPI
128. 
129. app = FastAPI()
130. 
131. 
132. @app.post("/process")
133. def process_message(message: str):
134.     return {
135.         "message": message.upper(),
136.         "length": len(message)
137.     }
```

---

24. Run FastAPI

All platforms:

138. uvicorn api:app --reload --port 8000

Expected:

```
Uvicorn running on http://127.0.0.1:8000
```

Open FastAPI documentation:

```
http://localhost:8000/docs
```

---

25. Test FastAPI

Open:

```
http://localhost:8000/docs
```

Select:

```
POST /process
```

Click:

```
Try it out
```

Enter:

```
hello world
```

Expected response:

```
{
  "message": "HELLO WORLD",
  "length": 11
}
```

---

26. Connect Streamlit to FastAPI

Replace `app.py` with:

```
import streamlit as st
import requests

st.title("Message Processor")

message = st.text_input("Enter your message")

if st.button("Process Message"):

    response = requests.post(
        "http://localhost:8000/process",
        params={"message": message}
    )

    result = response.json()

    st.subheader("Result")
    st.write(result["message"])
    st.write(f"Characters: {result['length']}")
```

---

27. Run the Complete Local Application

You need two processes.

Terminal 1 — FastAPI

```
uvicorn api:app --reload --port 8000
```

Terminal 2 — Streamlit

```
streamlit run app.py
```

Open:

```
http://localhost:8501
```

Enter:

hello data science

Expected:

HELLO DATA SCIENCE

Characters: 18

---

28. Message Flow

User
↓
Streamlit
↓
HTTP POST /process
↓
FastAPI
↓
Message Processing
↓
JSON Response
↓
Streamlit
↓
User

Example:

Input:
hello data science

FastAPI processing:

message.upper()
len(message)

Response:

```
{
    "message": "HELLO DATA SCIENCE",
    "length": 18
}
```

---

29. Basic Troubleshooting

If Streamlit cannot connect to FastAPI

Check whether FastAPI is running:

```
uvicorn api:app --reload --port 8000
```

Check FastAPI manually:

```
http://localhost:8000/docs
```

Check the API URL in app.py

It should be:

```
"http://localhost:8000/process"
```

Not:

```
"http://localhost:9000/process"
```

---

30. Commit FastAPI Version

```
git add .
```

```
git commit -m "Add FastAPI backend and messaging"
```

```
git push
```

---

31. Final Project Files

At this point:

```
message-processor/
│
├── app.py
├── api.py
├── requirements.txt
└── README.md
```

Docker files will now be added.

---

32. Dockerfile

Create a file named exactly:

```
Dockerfile
```

No `.txt` extension.

Content:

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

RUN chmod +x start.sh
```

```
229.
230.  EXPOSE 8501
231.  EXPOSE 8000
232.
233.  CMD ["./start.sh"]
```

---

33. start.sh

Create:

```
234.  start.sh
```

Content:

```
235.  #!/bin/bash
236.
237.  uvicorn api:app --host 0.0.0.0 --port 8000 &
238.
239.  streamlit run app.py \
240.      --server.address 0.0.0.0 \
241.      --server.port 8501
```

> Windows users do not need to execute `start.sh` directly. It runs inside the Linux-based Docker container.

---

34. Final requirements.txt

```
242.  streamlit
243.  fastapi
244.  uvicorn
245.  requests
```

---

35. Final Project Structure

```
message-processor/
│
├── app.py
├── api.py
├── requirements.txt
├── Dockerfile
├── start.sh
└── README.md
```

---

36. Git Commands — Complete Reference

Check repository:

```
git status
```

View changes:

```
git diff
```

Stage everything:

```
git add .
```

Stage a specific file:

```
git add app.py
```

Commit:

```
git commit -m "Your commit message"
```

View commits:

```
git log --oneline
```

View remote:

```
git remote -v
```

Push:

```
git push
```

Change branch name:

```
git branch -M main
```

---

37. Complete Local Application Flow

Before Docker:

```
Browser
   │
   ▼
Streamlit
localhost:8501
   │
   │ HTTP
   ▼
FastAPI
localhost:8000
   │
   ▼
Process Message
   │
   ▼
JSON Response
   │
   ▼
Streamlit
```

---

38. Docker Concepts

Application Code
+
Dockerfile
↓
Docker Image
↓
Docker Container
↓
Running Application

Simple terminology:

Dockerfile = Recipe

Image = Packaged application

Container = Running instance of the image

---

39. Final GitHub Workflow

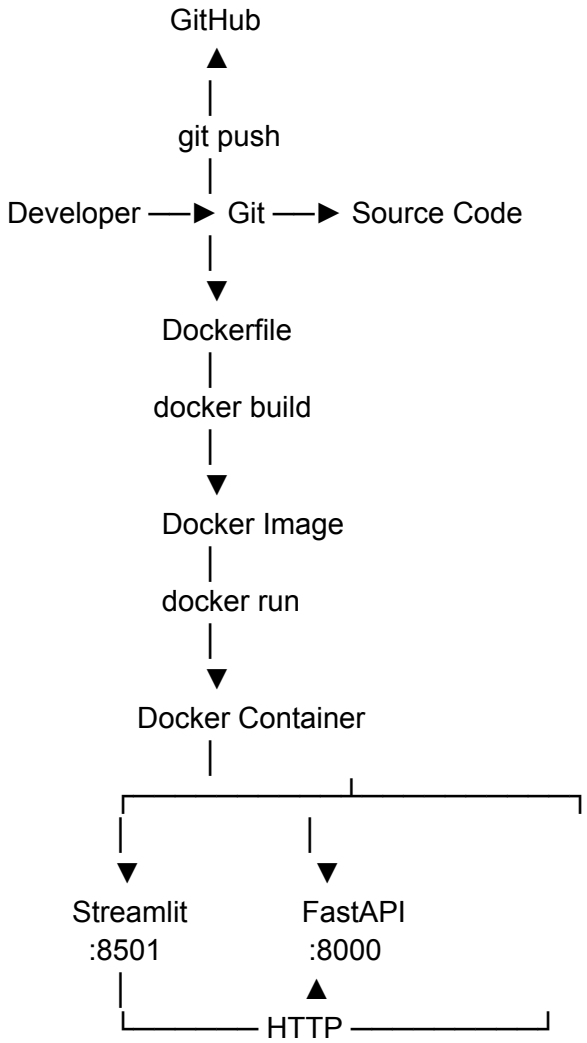
Local Code
↓
Git
↓
Commit
↓
GitHub

Git is the local source-control layer.

GitHub is the remote repository.

---

40. Final Application Architecture



---

41. Final Learning Checklist

By the end of the session, you should be able to:

- Create a basic Python project.
- Initialize a Git repository.
- Check Git status.
- Stage files.
- Commit changes.
- View Git history.
- Connect a local repository to GitHub.
- Push code to GitHub.
- Make and push subsequent changes.

- Create a basic Streamlit UI.
- Run Streamlit locally.
- Create a FastAPI backend.
- Create a `POST` API endpoint.
- Use FastAPI Swagger documentation.
- Connect Streamlit to FastAPI using HTTP.
- Trace a request from UI → API → UI.
- Perform basic API troubleshooting.
- Understand a Dockerfile.
- Understand image vs container.
- Package the application using Docker.
- Run the application locally in a container.
- Check container status and logs.
- Understand the complete:

330. Git
331. ↓
332. GitHub
333. ↓
334. Streamlit
335. ↓
336. FastAPI
337. ↓
338. Docker
339. ↓
340. Local Application

---

42. Important URLs Used During the Demo

Streamlit:

341. http://localhost:8501

FastAPI:

342. http://localhost:8000

FastAPI Swagger documentation:

343. http://localhost:8000/docs

---

43. Final Application Test

Enter:

344. Docker deployment

The application should return:

345. DOCKER DEPLOYMENT
346.
347. Characters: 17

At this point, the demonstration is complete.

Scope ends here: local Docker deployment.

No cloud deployment, Kubernetes, CI/CD, or production infrastructure is covered in this session.

348.